

Python — La visualisation de données

Document 60 — Matplotlib 3D (mplot3d)

Série « Visualisation » · la troisième dimension

Table des matières

1. Pourquoi la 3D, et ses limites

2. Première image : un axes 3D

2.1 Créer un Axes en projection 3d

2.2 Le troisième axe : `ax.set_zlabel`

3. Un graphique complet

4. Les types de tracés 3D

4.1 Nuage et courbe (`scatter`, `plot`)

4.2 Surface et maillage (`plot_surface`)

5. Le point de vue : angle et projection

6. Couleur et profondeur

7. Plusieurs vues et sauvegarde

8. 3D statique ou interactive ?

Récapitulatif

Questions de vérification

1. Pourquoi la 3D, et ses limites

Certaines relations n'ont de sens qu'en trois dimensions : une surface de volatilité (prix d'option selon le strike et l'échéance), une fonction de deux variables, un nuage à trois mesures. `mplot3d`, le sous-module 3D de Matplotlib, trace ces objets tout en restant dans l'écosystème déjà connu (document 10).

Mais la 3D se mérite : projetée sur un écran plat, elle souffre d'occlusion (des points en cachent d'autres) et d'ambiguïté de profondeur. Souvent, deux graphiques 2D bien choisis se lisent mieux qu'une seule vue 3D. On l'utilise quand la troisième dimension est vraiment porteuse de sens, pas pour l'effet.

Usage 3D pertinent	Exemple
Surface	Surface de volatilité : prix selon strike et échéance.
Fonction de 2 variables	$z = f(x, y)$, une nappe continue.
Nuage à 3 mesures	Trois indicateurs croisés sur un panier d'actifs.
Courbe paramétrique	Trajectoire dans un espace à trois axes.

Pourquoi ça compte. La 3D n'est pas « mieux » que la 2D : elle ajoute une dimension au prix de la lisibilité. Avant de la choisir, se demander si deux graphiques 2D ne diraient pas la même chose plus clairement. La 3D se justifie surtout pour les surfaces.

2. Première image : un axes 3D

2.1 Créer un Axes en projection 3d

Tout part du même fig, ax du document 10, à une différence près : on demande une projection « 3d ». L'Axes obtenu possède alors un troisième axe, z, et des méthodes de tracé en volume.

```
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")    # un Axes 3D
ax.scatter(xs, ys, zs)
plt.show()
```

Depuis Matplotlib 3.x, `add_subplot(projection="3d")` suffit ; l'ancien import explicite d'`Axes3D` n'est plus nécessaire (il reste mentionné au document 00 pour situer le sous-module).

2.2 Le troisième axe : `ax.set_zlabel`

Tout ce qu'on connaît en 2D se prolonge avec un z : `set_zlabel`, `set_zlim`, et un argument z dans les méthodes de tracé.

```
ax.set_xlabel("Strike")
ax.set_ylabel("Echeance")
ax.set_zlabel("Volatilite implicite")
```

Pourquoi ça compte. L'Axes 3D est le même objet Axes qu'en 2D, augmenté d'un axe z. Tout le vocabulaire du document 10 (`set_title`, `set_xlabel`, `legend`) reste valable : la 3D n'est pas une bibliothèque à part, c'est une projection de Matplotlib.

3. Un graphique complet

On habille un graphique 3D comme un 2D, en ajoutant le z. Le titre, les trois libellés d'axes et la légende suivent les mêmes méthodes.

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(strikes, echeances, vols, c=vols, cmap="viridis", label="IV")

ax.set_title("Surface de volatilité")
ax.set_xlabel("Strike")
ax.set_ylabel("Echeance (jours)")
ax.set_zlabel("Vol implicite")
ax.legend()
fig.tight_layout()
```

Méthode	Rôle
<code>set_zlabel(...)</code>	Libellé du troisième axe (z).
<code>set_zlim(...)</code>	Bornes de l'axe z.
<code>view_init(elev, azim)</code>	Angle de vue (élévation, azimuth).
<code>set_box_aspect(...)</code>	Proportions du cube de tracé.
<code>colorbar(...)</code>	Barre de couleur (échelle de z ou d'une mesure).

Le même geste, une dimension de plus. Chaque méthode 2D a son pendant en z. Si l'on sait construire un graphique au document 10, on sait l'étendre en 3D : il suffit d'ajouter la troisième coordonnée.

4. Les types de tracés 3D

4.1 Nuage et courbe (scatter, plot)

Les tracés les plus simples prolongent directement la 2D, avec un troisième tableau de coordonnées.

```
ax.scatter(xs, ys, zs)          # nuage de points 3D
ax.plot(xs, ys, zs)             # courbe / trajectoire 3D
```

4.2 Surface et maillage (plot_surface)

Le tracé 3D emblématique est la surface : une nappe $z = f(x, y)$. Il faut d'abord une grille de coordonnées, construite avec `numpy.meshgrid`, puis calculer z sur cette grille.

```
import numpy as np

x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)      # grille 2D de coordonnees
Z = np.sin(np.sqrt(X**2 + Y**2)) # z = f(x, y)

surf = ax.plot_surface(X, Y, Z, cmap="viridis")
fig.colorbar(surf)            # echelle de couleur
```

Méthode	Type	Quand l'utiliser
<code>scatter</code>	Nuage 3D	Trois mesures indépendantes.
<code>plot</code>	Courbe 3D	Trajectoire, courbe paramétrique.
<code>plot_surface</code>	Surface	Nappe continue $z = f(x, y)$.
<code>plot_wireframe</code>	Maillage	Surface en fil de fer, plus légère.
<code>contour3d</code>	Niveaux	Lignes de niveau projetées dans l'espace.

Pourquoi ça compte. `meshgrid` est le préalable de toute surface : il transforme deux axes 1D en une grille 2D de points où évaluer z . Oublier `meshgrid` et passer des vecteurs 1D à `plot_surface` est l'erreur de débutant n°1 en 3D.

5. Le point de vue : angle et projection

En 3D, l'angle de vue change tout : un même nuage peut sembler structuré ou informe selon l'orientation. On contrôle la caméra avec `view_init`.

```
ax.view_init(elev=30, azimuth=45) # elevation 30, azimuth 45 degrees
```

Paramètre	Effet
<code>elev</code>	Hauteur de la caméra (degrés au-dessus du plan).
<code>azim</code>	Rotation horizontale autour de l'axe z .
<code>roll</code>	Inclinaison de la caméra (Matplotlib récent).
<code>proj_type</code>	"persp" (perspective) ou "ortho" (orthographique).

Choisir un angle parlant. Pour un livrable, on fixe un `view_init` qui révèle la structure (le creux d'une surface, l'alignement d'un nuage). En exploration interactive, on fait tourner la vue à la souris pour trouver le bon angle, puis on le fige dans le code.

6. Couleur et profondeur

La couleur compense le manque de repères de profondeur d'un écran plat. La mapper sur le `z` (ou sur une quatrième mesure) aide l'œil à situer les points.

```
# couleur = hauteur z, pour lire le relief
p = ax.scatter(xs, ys, zs, c=zs, cmap="plasma")
fig.colorbar(p, label="Hauteur")

# transparence pour les nuages denses
ax.scatter(xs, ys, zs, alpha=0.3)
```

Levier	Rôle
<code>c= + cmap=</code>	Couleur des points/surface selon une mesure (souvent <code>z</code>).
<code>colorbar(...)</code>	Légende continue de l'échelle de couleur.
<code>alpha=</code>	Transparence : atténue l'occlusion d'un nuage dense.
<code>depthshade</code>	Assombrit les points lointains (indice de profondeur).

Pourquoi ça compte. En 3D, la couleur n'est pas qu'esthétique : elle restitue la profondeur que la projection écrase. Mapper la couleur sur le `z`, et baisser l'`alpha` sur les nuages denses, rend la scène nettement plus lisible.

7. Plusieurs vues et sauvegarde

Comme une seule vue 3D peut tromper, on en juxtapose souvent plusieurs, sous des angles différents — chacune dans son Axes 3D, via subplots.

```
fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax2 = fig.add_subplot(1, 2, 2, projection="3d")

ax1.plot_surface(X, Y, Z, cmap="viridis")
ax1.view_init(elev=30, azim=45)
ax2.plot_surface(X, Y, Z, cmap="viridis")
ax2.view_init(elev=80, azim=0)      # vue de dessus

fig.savefig("surface.png", dpi=200) # export comme en 2D
```

L'export est identique au 2D. Une fois la vue choisie, `fig.savefig` fonctionne exactement comme au document 10 : la 3D est rendue, puis aplatie en image. PNG pour l'écran, PDF/SVG vectoriels pour l'impression.

Pourquoi ça compte. Juxtaposer deux ou trois angles dans une même figure lève l'ambiguïté de profondeur bien mieux qu'une vue unique. C'est le réflexe des surfaces complexes : une vue de trois quarts et une vue de dessus.

8. 3D statique ou interactive ?

mplot3d produit une image 3D figée. Pour une vraie manipulation (tourner, zoomer librement), Plotly propose une 3D interactive en HTML. Le choix suit, une fois de plus, la sortie voulue.

Besoin	Outil
Image 3D pour un rapport	Matplotlib mplot3d (statique, vectoriel).
Exploration libre (rotation)	Plotly 3D (go.Surface, Scatter3d) en HTML.
Intégration au reste 2D	mplot3d : même écosystème que les documents 10–30.
Surface très dense, temps réel	PyQtGraph (GLSurfacePlotItem, OpenGL).

Le bon réflexe. Pour figer une surface dans un document, mplot3d, qui reste dans l'écosystème Matplotlib. Pour la faire tourner et l'inspecter, la 3D de Plotly (document 40). Pour une surface massive rafraîchie en direct, PyQtGraph (document 50).

***Pourquoi ça compte.** La 3D n'échappe pas au fil conducteur de la série : c'est la sortie qui décide. Image figée -> mplot3d ; exploration -> Plotly 3D ; temps réel -> PyQtGraph. mplot3d est simplement la voie statique, dans l'écosystème déjà maîtrisé.*

Récapitulatif

- 1. Le rôle.** Tracer en trois dimensions (surfaces, nuages, courbes), quand la troisième variable porte vraiment du sens.
- 2. La création.** `fig.add_subplot(projection="3d")` donne un Axes 3D ; tout le vocabulaire 2D se prolonge avec un `z`.
- 3. Les types.** `scatter` et `plot` prolongent la 2D ; `plot_surface` trace une nappe $z = f(x, y)$ à partir d'une grille `meshgrid`.
- 4. meshgrid.** Préalable obligatoire d'une surface : transforme deux axes 1D en grille 2D de coordonnées.
- 5. Le point de vue.** `view_init(elev, azim)` oriente la caméra ; un bon angle révèle la structure.
- 6. La couleur.** Mappée sur le `z` (`c = + cmap + colorbar`), elle restitue la profondeur que l'écran plat écrase.
- 7. Plusieurs vues.** Juxtaposer des angles lève l'ambiguïté ; l'export (`savefig`) est identique au 2D.
- 8. Le choix.** mplot3d (figé) vs Plotly 3D (interactif) vs PyQtGraph (temps réel) : la sortie décide.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Quand la 3D est-elle vraiment justifiée, et quelle est sa principale limite ?

Réponse. Quand la troisième dimension porte du sens : une surface (volatilité selon strike et échéance), une fonction $z = f(x, y)$, un nuage à trois mesures. Sa limite est l'occlusion et l'ambiguïté de profondeur : une fois projetée sur un écran plat ; souvent deux graphiques 2D se lisent mieux.

Q2. Comment crée-t-on un Axes 3D, et qu'a-t-il de plus qu'un Axes 2D ?

Réponse. Avec `fig.add_subplot(projection="3d")`. C'est le même objet Axes, augmenté d'un troisième axe `z` : `set_zlabel`, `set_zlim`, et un argument `z` dans les méthodes de tracé. Tout le vocabulaire du document 10 reste valable.

Q3. À quoi sert `numpy.meshgrid` avant un `plot_surface` ?

Réponse. À transformer deux axes 1D (`x` et `y`) en une grille 2D de coordonnées, où l'on évalue $z = f(X, Y)$. Sans `meshgrid`, `plot_surface` ne peut pas construire la nappe — c'est l'erreur n°1 des débutants en 3D.

Q4. Que contrôle `ax.view_init(elev, azim)` et pourquoi est-ce important ?

Réponse. L'angle de la caméra : `elev` (hauteur au-dessus du plan) et `azim` (rotation horizontale). En 3D, l'angle change radicalement la lisibilité ; on choisit un `view_init` qui révèle la structure, puis on le fige dans le code.

Q5. Pourquoi mapper la couleur sur le `z` en 3D ?

Réponse. Parce que la projection sur un écran plat écrase les repères de profondeur. Colorer les points ou la surface selon leur hauteur `z` (`c=zs`, `cmap`, `colorbar`) aide l'œil à situer les éléments. Baisser l'`alpha` atténue aussi l'occlusion des nuages denses.

Q6. Image 3D figée, exploration libre, surface temps réel : quel outil pour chacun ?

Réponse. Image figée pour un rapport : Matplotlib `mplot3d` (statique, vectoriel). Exploration libre avec rotation : Plotly 3D (document 40), en HTML interactif. Surface massive rafraîchie en direct : PyQtGraph (document 50). La sortie voulue décide, comme toujours dans la série.

Document suivant (70) : *Images et texte — afficher des images (Pillow, imshow), combiner texte et graphiques, et composer des overlays.*